

Course:	Code:	CO: CO5
Duration:	Total Marks: 100	Reg No / Name :

COURSE OUTCOME COVERED

CO5: *Analyze vulnerabilities in Windows and Linux systems and evaluate security weaknesses in messaging, web, and mobile applications using appropriate exploitation techniques and hacking tools. (BT5)*

Instructions: Answer all questions. Each question carries 20 marks.

- 1. A mid-sized financial organization operates several legacy Windows servers where multiple RPC services remain exposed due to delayed patching cycles caused by operational constraints. Recently, abnormal traffic patterns and unauthorized connection attempts have been observed within the internal network. As a security analyst, evaluate how attackers could exploit these RPC vulnerabilities to gain unauthorized access and escalate privileges. Assess the level of risk associated with delaying patches in such environments and justify whether immediate patch deployment or controlled maintenance scheduling is more appropriate. Recommend suitable mitigation strategies to reduce exposure without affecting business continuity.**

Legacy Windows Servers with Exposed RPC Services

Remote Procedure Call (RPC) is a Windows communication mechanism that allows applications and services to communicate across systems. When RPC services are exposed on legacy Windows servers and security patches are delayed, attackers may exploit known vulnerabilities to obtain unauthorized access.

How attackers could exploit RPC vulnerabilities

An attacker who discovers an exposed and vulnerable RPC service can perform reconnaissance to identify the service version and determine whether it is affected by a known vulnerability. If a vulnerable RPC implementation is present, an attacker may send specially crafted requests that cause unexpected behavior, such as memory corruption or unauthorized execution of code.

A successful attack could provide an attacker with an initial foothold on the Windows server. Depending on the privileges of the vulnerable service, the attacker may then be able to access files, execute commands, create accounts, modify configurations, or establish persistence.

Attackers may also attempt privilege escalation after gaining initial access. If the compromised server contains additional local vulnerabilities or improperly configured privileges, the attacker could potentially move from a low-privileged account to an administrative or SYSTEM-level context.

The observed abnormal traffic and unauthorized connection attempts are therefore important indicators that the exposed RPC services require immediate investigation. The assessment specifically identifies delayed patching and unauthorized connection attempts as the major concerns.

Risk of delaying patches

The risk is high, particularly for legacy systems because:

- Known vulnerabilities may already have publicly available exploitation techniques.
- Attackers can scan networks for exposed RPC services.
- A compromised server may become a platform for attacking other internal systems.
- Administrative privileges could result in complete system compromise.
- Sensitive financial information may be exposed.
- Malware or ransomware could spread through the internal network.
- Delayed patching increases the time available for attackers to exploit the vulnerability.

Immediate patching vs controlled maintenance

Immediate risk reduction is preferable, but patch deployment should be performed in a controlled manner to avoid business disruption.

A suitable approach is:

1. Identify the affected servers and RPC services.
2. Verify whether the vulnerabilities are actively exploitable.
3. Apply temporary network restrictions immediately.
4. Test patches on a representative server where possible.
5. Deploy security updates according to business-criticality.
6. Monitor systems after patching.
7. Maintain a rollback or recovery plan.

If immediate patching is technically impossible, temporary isolation and access restriction should be implemented until maintenance can be performed.

Mitigation strategies

- Apply current Microsoft security updates.
- Restrict RPC access using Windows Firewall and network firewalls.
- Allow RPC traffic only from authorized systems and networks.
- Disable unnecessary RPC services.
- Segment legacy servers from critical systems.
- Use network monitoring and IDS/IPS to detect suspicious RPC traffic.

- Implement vulnerability scanning and patch-management procedures.
- Enable strong authentication and least-privilege access.
- Maintain regular backups.
- Monitor Windows event logs for unusual authentication and service activity.

Conclusion

Delaying patches on vulnerable RPC services creates significant security exposure. A risk-based, controlled patching strategy combined with immediate network restrictions provides a better balance between security and business continuity. Critical vulnerabilities should not remain exposed simply because of operational concerns.

2. **A web application used for online transactions suffers from multiple vulnerabilities, including cross-site scripting (XSS), cross-site request forgery (CSRF), and weak session management practices. Users report unauthorized transactions and session hijacking incidents. Evaluate how attackers could combine these vulnerabilities to execute complex attacks affecting user accounts. Assess the risks associated with poor session handling and lack of input validation. Justify the need for a layered security approach and recommend comprehensive mitigation strategies to secure the application.**

XSS, CSRF and Weak Session Management in an Online Transaction Application

The application contains three serious weaknesses: Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and weak session management. These vulnerabilities can become considerably more dangerous when attackers combine them. The scenario reports unauthorized transactions and session hijacking, directly demonstrating the potential impact.

How attackers could combine the vulnerabilities

An attacker may first identify an input field that does not properly validate or encode user-controlled data. If the application is vulnerable to XSS, malicious script content could be executed in another user's browser.

If the victim is already authenticated, the malicious script could potentially interact with the application's pages and functions using the victim's active session.

Weak session management increases the impact. If session identifiers are predictable, insufficiently protected, long-lived, or improperly invalidated, an attacker may be able to hijack a user's authenticated session.

CSRF can then be used to cause an authenticated browser to submit an unwanted transaction. For example, a victim who is logged into an online transaction application could unknowingly trigger an unauthorized state-changing request if the application does not properly verify that the request originated from the legitimate application interface.

Thus, the vulnerabilities can reinforce one another:

XSS → victim browser compromise → abuse of authenticated session → unauthorized actions
and

Weak session management + CSRF → unauthorized authenticated requests

Risks of poor session handling

Poor session management can lead to:

Session hijacking.
Account takeover.
Unauthorized transactions.
Exposure of personal information.
Privilege abuse.
Persistent authentication after logout.
Session fixation attacks.
Increased impact of XSS and CSRF.
Risks of inadequate input validation

Poor input validation allows attackers to submit unexpected or malicious data. In the case of XSS, this can result in malicious content being interpreted by the victim's browser.

Possible consequences include:

Theft of sensitive information.
Unauthorized actions.
Modification of displayed content.
User redirection.
Account compromise.
Damage to customer trust.
Need for layered security

No single control can completely protect the application. A layered approach is required because an attacker may bypass one defensive mechanism and exploit another weakness.

Important security layers include:

Secure input validation.
Context-aware output encoding.
CSRF protection.
Secure session management.
Authentication and authorization controls.
Secure cookies.
Content Security Policy.
Server-side validation.
Security logging and monitoring.
Regular vulnerability testing.

Mitigation strategies

For XSS:

Validate input on the server.
Encode output according to its context.
Avoid unsafe dynamic HTML generation.
Implement Content Security Policy.
Sanitize HTML where HTML input is genuinely required.

For CSRF:

Use unpredictable CSRF tokens.
Validate tokens on state-changing requests.
Use appropriate SameSite cookie settings.
Require additional authentication for high-risk transactions.

For session management:

Generate strong random session identifiers.
Use HTTPS.
Set Secure, HttpOnly, and appropriate SameSite cookie attributes.
Expire inactive sessions.
Regenerate session IDs after authentication.
Invalidate sessions after logout.
Avoid placing session identifiers in URLs.

Conclusion

The combination of XSS, CSRF, and weak session management can allow attackers to compromise authenticated user sessions and perform unauthorized actions. Therefore, input validation, secure session handling, CSRF protection, HTTPS, and continuous security testing must operate together.

- 3. An enterprise uses an outdated IIS server to host multiple internal and external applications, and recent vulnerability scans indicate several unpatched critical flaws. Despite warnings, the organization delays updates due to fear of downtime and service disruption. Assess how attackers could exploit these vulnerabilities to gain unauthorized access or execute remote code. Evaluate the risks associated with delaying security updates in such environments and justify whether immediate patching or temporary isolation is more appropriate. Recommend strategies to ensure both security and service availability.**

Outdated IIS Server with Critical Unpatched Vulnerabilities

Internet Information Services (IIS) is a Windows web-server platform used to host web applications and services. An outdated IIS server containing multiple critical unpatched vulnerabilities presents a significant security risk. The assessment scenario specifically identifies unpatched critical flaws and organizational concerns about downtime.

How attackers could exploit the vulnerabilities

An attacker may first perform reconnaissance against the IIS server to identify:

- IIS version.

- Exposed ports.
- Hosted applications.
- Web directories.
- Enabled services.
- Known vulnerabilities.

If the server contains a vulnerable IIS component or web application, an attacker may attempt to exploit it.

Depending on the vulnerability, exploitation could potentially result in:

- Unauthorized access.
- Information disclosure.
- Authentication bypass.
- Web application compromise.
- Unauthorized file access.
- Remote code execution.
- Server takeover.

Remote code execution is particularly serious because it may allow an attacker to execute commands under the security context of the vulnerable service.

After obtaining access, an attacker could attempt privilege escalation, establish persistence, access application databases, steal credentials, or use the server to attack other systems.

Risks of delaying security updates

Delaying updates creates several risks:

- Known vulnerabilities remain exploitable.
- Attackers have additional time to discover and exploit weaknesses.
- Publicly documented vulnerabilities can be easier to target.
- Compromise of an externally accessible server may provide an entry point into the organization.
- Customer and organizational information could be exposed.
- Website or application availability could be affected.
- Malware or ransomware could be introduced.
- Recovery costs may become greater than planned maintenance costs.

Immediate patching vs temporary isolation

For **critical vulnerabilities**, immediate risk reduction is necessary.

If testing confirms that patches can safely be deployed, the preferred approach is:

Backup → Test → Patch → Verify → Monitor

However, if immediate patching could cause unacceptable service disruption, temporary isolation should be used as a short-term compensating control.

Isolation may include:

- Restricting access through a firewall.
- Removing unnecessary public exposure.
- Allowing traffic only from trusted sources.
- Moving affected applications behind a protected reverse proxy.
- Disabling vulnerable services where practical.

Temporary isolation should **not become a permanent substitute for patching**.

Strategies for security and availability

A practical maintenance strategy includes:

1. Identify critical IIS vulnerabilities.
2. Classify systems according to business importance.
3. Take verified backups.
4. Test security updates in a staging environment.

5. Schedule maintenance during low-traffic periods.
6. Deploy patches in phases.
7. Monitor application performance.
8. Maintain rollback procedures.
9. Use firewall restrictions as compensating controls.
10. Perform vulnerability scans after patching.

Conclusion

The organization should not indefinitely delay critical security updates because of downtime concerns. **Immediate patching is preferred**, while temporary isolation can reduce exposure when patching cannot be performed immediately. This approach provides both security and service availability.

4. **A mobile application used by customers stores sensitive information such as login credentials and personal data in plaintext within the device storage. Additionally, the application communicates with backend servers over unsecured channels. Evaluate how attackers could exploit these weaknesses through device compromise or network interception. Assess the potential impact on user privacy and organizational reputation. Justify the need for encryption and recommend secure storage and communication techniques.**

Mobile Application Storing Sensitive Data in Plaintext and Using Unsecured Communication

The mobile application stores sensitive information such as login credentials and personal data in plaintext and communicates with backend servers through unsecured channels. These weaknesses create serious privacy and security risks. The assessment specifically asks for analysis of device compromise, network interception, privacy impact, encryption, and secure storage/communication techniques.

Exploitation through device compromise

If an attacker obtains unauthorized access to the mobile device, plaintext application data may be accessible through local storage.

Potentially exposed information may include:

- Usernames.
- Passwords.
- Authentication tokens.
- Personal information.
- Application configuration data.
- Transaction-related information.

A compromised or rooted device can increase the attacker's ability to inspect application files and local storage.

Storing passwords directly on the device is particularly dangerous because an attacker who obtains them may attempt to reuse those credentials against the application or other services.

Network interception

When communication occurs over an unsecured channel, attackers positioned between the mobile application and server may be able to observe or manipulate network traffic.

Potential risks include:

- Credential exposure.
- Personal-data interception.
- Session-token theft.
- Modification of requests.
- Manipulation of application responses.
- Man-in-the-middle attacks.

For example, if authentication information is transmitted without adequate transport encryption, an attacker monitoring the communication path may obtain sensitive information.

Privacy impact

Exposure of customer information can cause:

- Identity-related fraud.
- Account compromise.
- Loss of customer trust.
- Privacy violations.
- Financial losses.
- Regulatory consequences.
- Damage to organizational reputation.

A security incident involving customer information can therefore have both technical and business consequences.

Importance of encryption

Encryption protects information by making it unreadable to unauthorized parties.

Two important areas are:

Data at rest:

Sensitive information stored on the device should be encrypted and protected using platform-supported secure mechanisms.

Data in transit:

Communication between the mobile application and backend server should use modern HTTPS/TLS rather than plaintext communication.

Secure storage techniques

The application should:

- Avoid storing passwords in plaintext.
- Use platform-provided secure storage mechanisms.
- Protect authentication tokens appropriately.
- Minimize locally stored sensitive information.
- Encrypt sensitive data when local persistence is required.
- Apply appropriate access controls.
- Avoid storing secrets in easily accessible application files.

Passwords should generally be handled using secure server-side password hashing rather than storing the original password.

Secure communication techniques

The application should:

- Use HTTPS/TLS for all sensitive communication.
- Validate server certificates correctly.
- Protect authentication tokens.

- Reject insecure connections.
- Avoid transmitting sensitive information through unencrypted protocols.
- Apply appropriate server-side authentication and authorization.
- Monitor communication failures and suspicious activity.

Conclusion

Plaintext storage and unsecured communication create significant security and privacy risks.

Secure local storage, encryption at rest, HTTPS/TLS, proper authentication-token handling, and minimal local data storage should be implemented to protect both users and the organization.

5. **A system administrator disables the Windows firewall temporarily to troubleshoot connectivity issues but forgets to re-enable it. Within a short period, multiple unauthorized access attempts are detected. Evaluate how attackers could exploit this temporary exposure to gain access to critical services. Assess the risks associated with mismanagement of security controls and justify the importance of enforcing automated policies. Recommend administrative safeguards to prevent such incidents.**

Windows Firewall Disabled and Not Re-enabled

A Windows firewall provides an important layer of protection by controlling network traffic to and from a system. In this scenario, the administrator disables the firewall for troubleshooting and forgets to re-enable it. Shortly afterward, unauthorized access attempts are detected. The scenario emphasizes the risks of mismanaged security controls and the importance of automated policies.

How attackers could exploit the exposure

When the firewall is disabled, network services that were previously protected by firewall rules may become reachable.

An attacker could:

1. Discover the exposed host through network scanning.
2. Identify reachable services and open ports.
3. Determine which services are running.
4. Attempt authentication against exposed services.
5. Exploit vulnerable services or weak configurations.
6. Attempt privilege escalation after gaining access.
7. Establish persistence or move laterally within the network.

For example, a service that should only be accessible from a restricted management network could become reachable from a larger network when firewall protection is removed.

Risks of mismanagement of security controls

The incident demonstrates that security controls can become ineffective because of administrative mistakes rather than technical vulnerabilities.

Major risks include:

- Unauthorized network access.
- Exposure of internal services.
- Exploitation of vulnerable applications.
- Credential attacks.
- Malware infection.
- Lateral movement.

- Data theft.
- Service disruption.

The problem is particularly serious because the firewall remained disabled after troubleshooting was completed.

Importance of automated policies

Automated security policies reduce dependence on human memory.

For example:

- Firewall configuration can be centrally managed.
- Security baselines can detect configuration changes.
- Endpoint management systems can enforce firewall status.
- Monitoring systems can generate alerts when protection is disabled.
- Administrative actions can be logged and audited.

Automation helps ensure that temporary troubleshooting changes do not become permanent security weaknesses.

Administrative safeguards

The organization should implement:

1. Change-management procedures

Firewall changes should be documented, approved, and assigned a defined duration.

2. Time-limited exceptions

Temporary firewall changes should automatically expire after the required troubleshooting period.

3. Centralized management

Use centralized security-policy management to enforce firewall configurations across systems.

4. Monitoring and alerting

Generate alerts whenever the firewall is disabled or security policies are modified.

5. Least privilege

Administrators should receive only the privileges necessary to perform their tasks.

6. Configuration auditing

Regularly verify firewall status and compare configurations against approved security baselines.

7. Incident response

Investigate unauthorized access attempts and determine whether any compromise occurred during the period of exposure.

8. Documentation

Maintain records of troubleshooting changes, responsible administrators, start times, and restoration times.

Recommended response to the incident

The administrator should:

1. Re-enable the Windows firewall.
2. Verify that the correct firewall rules are active.
3. Review security and firewall logs.
4. Identify the source and nature of unauthorized attempts.
5. Check for suspicious accounts, processes, or configuration changes.
6. Scan the system for malware.
7. Review other systems for similar configuration problems.
8. Implement automated enforcement to prevent recurrence.

Conclusion

Disabling a firewall temporarily can create a serious security gap if the control is not restored.

The organization should combine **least privilege, change management, automated policy enforcement, continuous monitoring, and configuration auditing** to prevent administrative errors from becoming security incidents.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Mark
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem	
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts	
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach	
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution	
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand	